

자구&알고 스터디

집합과 맵
우선순위 큐

맵

- Key – Value로 데이터를 저장하는 자료구조
 - Key : 중복되지 않는 값
 - Value : Key값에 할당되는 값
- 원하는 정보 “value”에 서로 안 겹치는 이름 “key”를 부여하는 것과 똑같이 생각하면 편함.
- 반대로, 특정 정보 “key”에 또다른 정보를 연결 시키는 것처럼 생각하면 편함.

맵

- C++에서 종류 3가지
 - map : key 값들이 오름차순으로 정렬되어 있고, key 값들을 오름차순을 유지
 - unordered_map : 해쉬 테이블
 - multimap : 중복 가능
- 맵의 의의: 두 정보를 연관 시켜 정보를 저장할 수 있다.
 - 예시) 음식 주문 정보 key:주문한 사람, value: 주문한 음식
 - 철수: 파스타
 - 민수: 파스타
 - 영희: 삼겹살

맵

1. 선언

```
map<string, int> student;
```

1-1. 선언 및 초기화

```
map<string, int> student = {{"철수", 123456}, {"민수", 234567}};
```

2. 값 접근

```
cout << "철수 학번: " << student["철수"] << endl;
```

맵

3. 삽입

```
student["지수"] = 456789; // 새로운 원소 삽입  
student.insert({"하나", 567890}); // insert로도 가능
```

4. 삭제

```
student.erase("민수"); // key가 "민수"인 원소 삭제
```

5. 원소 체크

```
Student.count("민수"); // key가 "민수"인 원소가 있으면 1, 없으면 0  
반환
```

맵

- 예제 문제 : <https://www.acmicpc.net/problem/17219>
 - C++ 예시 답안: <http://boj.kr/f09890ec769c4f8aa99735c633d1ceb8>

집합

- 영문명칭이 Set이라 셋이라 부르기도 함
- 중복 없는 정보의 모음
- 이럴 때 편리하다
 - 중복 없는 원소 모음을 관리할 때
 - 특정 값 존재 여부를 자주 확인할 때
 - (C++ 한정) 원소를 정렬된 상태로 관리하고 싶을 때.
- 여담 (C++ 한정)
 - 원소가 정렬된 상태가 아니어도 될 때 : unordered_set 사용
 - 여러 원소가 중복해서 들어올 때 : multiset 사용

집합

1. 선언

```
set<int> s;
```

1-1. 선언 + 초기화

```
set<int> s = {1,2,4,4}; // 중복 → 하나만 남김
```

```
// 삽입
```

```
s.insert(5);
```

```
s.insert(5); // 중복 → 무시됨
```

```
s.insert(3);
```


집합

```
// 탐색  
if (s.find(3) != s.end()) {  
    cout << "3은 집합에 포함 되어있습니다." << endl;  
}
```

```
// 삭제  
s.erase(1);
```

집합

- 예제 문제 : <https://www.acmicpc.net/problem/14425>
- C++ 예시 답안: <http://boj.kr/ed656bb2f5bb4c829844d6e20c0a7594>

우선순위 큐

- 일반 큐와 달리, 들어온 순서대로 정보를 저장하는 것이 아니라, 우선순위를 기준으로 정보를 저장하는 자료구조
- 우선순위 큐는 새로운 정보를 삽입, 삭제할 때 우선순위 기준으로 정보를 재정렬 해야함.
- 단, 우선순위 큐를 쓰면서 중요한 것은 가장 앞에 있는 (또는 가장 위에 있는) 정보가 우선순위가 가장 높은 것이어야 한다는 점!
 - 이 말은 다르게 말하면 어떤 자료구조 (트리, 리스트 등등)을 써도, 새로운 정보를 삽입, 삭제할 때 우선순위가 가장 앞 (또는 가장 위)에 배치할 수 있도록 유지시킬 수만 있다면 우선순위 큐로 사용 가능

우선순위 큐

- 우선순위 큐에서 우선순위는 구현하는 방식에 따라 다름
- 가장 기본적인 방법은 최솟값 최댓값 기준으로 우선순위를 정하는 것
- 필요에 따라 우선순위 정하는 함수 만들어 활용 가능

우선순위 큐

- 구현 방법: 리스트
 - 리스트를 선언
 - 리스트에 정보가 추가/삭제 될 때마다 리스트 정렬.
- 구현 방법: 힙 구현
 - 학교 수업에서 배움.
 - C++에서는 힙을 구현 할 수 있는 함수들을 제공하긴함 ([링크](#))
- 구현 방법: `std::priority_queue` 활용
 - 가장 보편적인 방법
 - 내부적으로 힙을 활용하여 구현됨
 - 우선순위 정하는 함수 직접 구현 및 적용 가능

우선순위 큐

1. 선언

- `priority_queue<int> pq;`
 - 기본 `priority_queue`는 최대 힙 (큰 값이 먼저 나옴)
- `priority_queue<int, vector<int>, greater<int>> pq;`
 - 최소 힙 (작은 값이 먼저 나옴)

2. 삽입 (push)

- `pq.push(10);`

우선순위 큐

3. 접근 (top) — 항상 가장 우선순위 높은 원소 확인

- `pq.top();` // 우선순위 가장 높은 원소 반환

4. 삭제 (pop) — top을 제거

- `pq.pop();`

우선순위 큐

- 예제 문제 : <https://www.acmicpc.net/problem/11279>
- C++ 예시 답안 : <http://boj.kr/274b68344ec649b69253dc000caa7248>

복습 문제

- 집합과 맵 : <https://www.acmicpc.net/step/49>
- 우선순위 큐 : <https://www.acmicpc.net/step/13>